Step 2 — The Planner and the RAG Search — Hints

> Back to the step · Hint 1 · Hint 2 · Solution

- Hint 1 — the planner is one structured call
- Hint 2 — search is a closure that carries the source

Hint 1 — the planner is one structured call

"Plan first" (Doc07) is a single structured-output call:

```
make_plan(model, question):
    prompt = "break this question into 3-5 focused sub-questions ..."
    chain = prompt | model.with_structured_output(SubQuestions)
    return chain.invoke({"question": question})
```

That's the whole planner — the model returns a `SubQuestions` with a list you loop over. Writing the plan down before anything runs is what makes the agent reviewable.

Hint 2 — search is a closure that carries the source

`build_source_store()` chunks and embeds `data/sources/*.md`, keeping each chunk's **filename** in `metadata["source"]` — that filename is the citation. Then `make_search` returns a closure the executor calls:

```
def make_search(collection):
    def search(query):
        results = collection.query(query_embeddings=[embed(query)], n_results=1)
        text = results["documents"][0][0]
        source = results["metadatas"][0][0]["source"]   # the filename
        return text, source
    return search
```

The executor calls `search(sub_question)` and never imports Chroma — which is exactly why Step 1 could test it with a fake `search`. Keep just the file name (`source.split("/")[-1]`), not

the full path, so citations read `rag.md`. The whole file set is in the Solution.